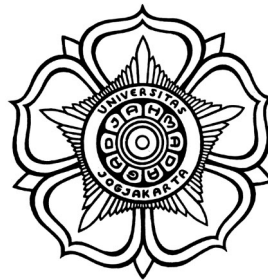


**ASSIGNMENT ON LECTURE
DATABASE COURSE**

RELATIONAL MODEL AND QUERY ON IT

Instructor:
Khabib Mustofa



submitted by
Ahmad Kailani
21/530323/SPA/01010

**UNDERGRADUATE PROGRAM IN COMPUTER SCIENCE
DEPARTMENT OF COMPUTER SCIENCES AND ELECTRONICS
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

CONTENTS

Cover Page	i
I Guidelines and Assignment Format	1
1.1 Assignment Guidelines	1
1.2 Assignment Content Details	2
1.2.1 Case Domain	2
1.2.2 Output/Outcome of the Assignment	3
1.3 Sample Problem and Its Solution	4
II Results and Discussion	7
2.1 Entity Relationship Diagram	7
2.2 Schema of Tables in the Database	8
2.3 Business Questions and Solutions Using SQL	8
2.3.1 Question number 1	8
2.3.2 Question number 2	8
2.3.3 Question number 3	9
2.3.4 Question number 4	9
2.3.5 Question number 5	10
REFERENCES	11

LIST OF TABLES

LIST OF FIGURES

2.1 ER Diagram of the Academic Database	7
---	---

CHAPTER I

Guidelines and Assignment Format

Academic Integrity

All work submitted must be the result of the student's own thinking and work. Copying code or reports from other sources without clear attribution is a violation of academic integrity and will be subject to action in accordance with university regulations.

1.1 Assignment Guidelines

The assignment concerns the Relational Model and Queries on the Relational Model:

- Relational Algebra
- *Structured Query Language (SQL)*

The assignment answers are packaged in one chapter (chapter 2), whose sections have been provided in this template. The content of that chapter may be expanded as needed, but not reduced.

Deliverables

The assignment consists of four parts

1. Translate business requirements and processes into an ER diagram (*Entity-Relationship Diagram*). The proposal must include an explanation of the ERD creation process
2. Design and propose a relational database schema that aligns with the proposed ERD
3. Given a query, propose a representation of the query in relational algebraic expressions.
4. Propose a solution to the problem in the form of an SQL queries. If there are more than one possible query (with dif-

ferent operations or clauses), alternative queries must also be provided

Translated with DeepL.com (free version)

Assignment Conditions

1. The work may be done in groups with a maximum of 4 members.
2. Each group member **must** contribute adequately, and master the material produced by their group.
3. The work must be presented *clearly, completely, and neatly, including the use of formal language that is easy to understand*
4. The work must come from the team members' own thinking, and be agreed upon by all members

1.2 Assignment Content Details

1.2.1 Case Domain

It is desired to manage the academic data of a university with the following constraints:

Process Constraints

- Every academic activity needs to be recorded transparently and preserve chronology/history
- The data storage design accommodates the following matters and processes:
 - A student is allowed to retake a course without any limit on the number of times
 - The course grade (if retaken) used for the transcript is the **grade from the last attempt**
 - Courses are either COMPULSORY or ELECTIVE.

- A course may have other courses as prerequisites
- A lecturer may teach a course in a semester in more than one class (parallel)
- Some courses may be offered in even or odd semesters
- A class for a course is taught entirely by one lecturer (not team teaching)
- The course grades and weights given are

Grade	Weight	Grade	Weight
A	4.00	B/C	2.50
A-	3.75	C+	2.25
A/B	3.50	C	2
B+	3.25	C/D	2.5
B	3.00	D	1.00
B-	2.75	D	0.00

- The formula for calculating GPA from n courses is:

$$GPA = \frac{\sum_{i=1}^n \text{credits}_i \cdot \text{weight}_i}{\sum_{i=1}^n \text{credits}_i}$$

1.2.2 Output/Outcome of the Assignment

The following are some **minimal** output/outcome to be addressed.

Deliverables

1. An E-R diagram (ERD) that matches the points above! (starting from identification: entities, relations, cardinalities, etc.)

2. A complete schema of the database tables that matches the ERD created; for example in the form

```
Student(nim, name, gender, studyProgram, ...)
Lecturer(nip, ...)
```

And **also SQL DDL commands** for defining the tables above (complete with all *constraints*)!

3. DML commands for the questions below:

- (a) **List of elective course names and the courses that are their prerequisites (only name and course code information is sufficient). Courses that do not have prerequisites should still be displayed!**
- (b) **List of elective courses taken by Computer Science students of the '2024' cohort in the odd semester of academic year 2026/2027!**
- (c) **List of favorite elective courses for Computer Science students, namely the three courses with the most enrollees in the last three years!**
- (d) **List of students who have a Semester Grade Point Average for the odd semester of 2025/2026 of no less than 3.25! (understand the difference between Semester GPA and Cumulative GPA)**
- (e) **List of students and the Cumulative GPA they have obtained to date! (courses that have been retaken only need to be counted once)**

4. Relational algebra expression for each given query!

For each query above, the steps for working on it need to be described:

1. identify the tables that need to be involved
2. determine the join criteria and join type: INNER JOIN, NATURAL JOIN, LEFT/RIGHT JOIN OR OUTER JOIN
3. Valid SQL commands. If a VIEW is needed as a temporary table, please also write it out in order.
4. if there is more than one *equivalent* SQL query, it will be a bonus if you can also write that query!

1.3 Sample Problem and Its Solution

Suppose the following query is given:

Display the list of elective courses that have never been taken/attended by students

then an example answer is as follows:

Query number 1

1. **Tables needed in the query:** COURSE (Mk), COURSEENROLLMENT (Pk)
2. if using the SET DIFFERENCE operation there is no join process, so there is no join criterion. As an alternative, LEFT JOIN can be used, with the criterion `Mk.courseId= Pk.courseId`

3. **The required query**

```
SELECT * FROM Course Mk WHERE (type='E') and courseId NOT
IN
(SELECT courseId FROM CourseEnrollment Pk)
```

Listing I.1: SET DIFFERENCE approach

4. Equivalent SQL alternatives

```
SELECT Mk.* FROM Course Mk LEFT JOIN CourseParticipant Pk
ON (Mk.courseId=Pk.courseId)
WHERE Mk.type='E' and Pk.courseId is NULL
```

Listing I.2: Using LEFT JOIN

```
SELECT * FROM Course Mk WHERE (type='E') and
NOT EXISTS
(SELECT courseId FROM CourseEnrollment Pk
WHERE Pk.courseId=Mk.courseId)
```

Listing I.3: Using EXISTS

5. Proposed Relational algebra expression

$$(\pi_{\text{courseId}}(\sigma_{\text{type}='E'}(\text{Course}))) - \pi_{\text{courseId}}(\text{Course}) \bowtie \text{Course}$$

$$\begin{aligned} DIFF1 &\leftarrow \pi_{\text{courseId}}(\sigma_{\text{type}='E'}(\text{Course})) \\ DIFF2 &\leftarrow (DIFF1 - \pi_{\text{courseId}}(\text{Course})) \\ DIFF2 &\bowtie \text{Course} \end{aligned}$$

CHAPTER II

Results and Discussion

Group Members

1. Nashwan Hafid Andayuri, 25/554944/PA/23251
2. Mhs 2, NIM:
3. Mhs 3, NIM:

2.1 Entity Relationship Diagram

From the analysis of the business processes and the given constraints, this section can explain several things, including: *cardinality mapping*, relevant attributes and keys, etc.

The ER diagram for the Academic Database is shown in Figure 2.1

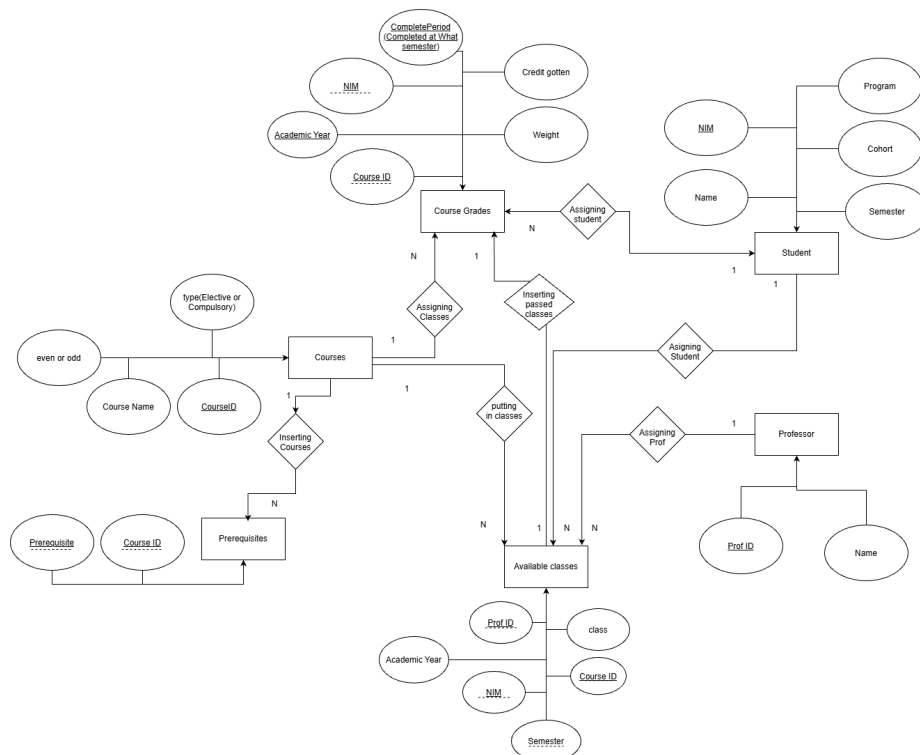


Figure 2.1: ER Diagram of the Academic Database

2.2 Schema of Tables in the Database

Table Schema in the Academic Database

- students(nim, name, semester, cohort, program)
- professors(prof id, name)
- courses(course name, course id, period, type, academic year)
- prerequisites(prerequisite, course id)
- AvailableClasses(prof id, nim, class, course id, semester, academic year)
- CourseGrades(course id, nim, academic year, completed period, credits, weight)
-

```
CREATE TABLE professors (
  prof_id VARCHAR(20) PRIMARY KEY, name VARCHAR(100) NOT NULL);
```

```
CREATE TABLE courses ( course_id VARCHAR(20) PRIMARY KEY, course_name
```

```
CREATE TABLE prerequisites ( prerequisite VARCHAR(20),
  course_id VARCHAR(20), PRIMARY KEY (prerequisite, course_id), FOREIGN KEY (pr
```

```
CREATE TABLE AvailableClasses (
  prof_id VARCHAR(20), nim VARCHAR(20), class VARCHAR(20), course_id VARCHAR(
```

```
CREATE TABLE CourseGrades ( course_id VARCHAR(20), nim VARCHAR(20), acade
```

```
CREATE TABLE professors ( prof_id VARCHAR(20) PRIMARY KEY, name VARCHAR
```

```
CREATE TABLE courses ( course_id VARCHAR(20) PRIMARY KEY, course_name VAR
```

```
CREATE TABLE prerequisites ( prerequisite VARCHAR(20),
  course_id VARCHAR(20), PRIMARY KEY (prerequisite, course_id), FOREIGN KEY (pr
```

```
CREATE TABLE AvailableClasses (
  prof_id VARCHAR(20), nim VARCHAR(20), class VARCHAR(20), course_id VARCHAR(
```

```
CREATE TABLE CourseGrades ( course_id VARCHAR(20), nim VARCHAR(20), acade
```

2.3 Business Questions and Solutions Using SQL

2.3.1 Question number 1

List of elective course names and the courses that are their pre-requisites (only name and course code information is sufficient). Courses that do not have prerequisites should still be displayed!

1. Tables needed to answer the query:
2. Join criteria (if any)
3. SQL command according to the question

```
SELECT * FROM course where type='E'
```

4. Alternative SQL commands (if any) with adequate explanation

2.3.2 Question number 2

List of elective courses taken by Computer Science students of the '2024' cohort in the odd semester of academic year 2026/2027!

1. Tables needed to answer the query:
2. Join criteria (if any)
3. SQL command according to the question

```
SELECT
FROM
```

4. Alternative SQL commands (if any) with adequate explanation

2.3.3 Question number 3

List of favorite elective courses for Computer Science students, namely the three courses with the most enrollees in the last three years!

1. Tables needed to answer the query:

2. Join criteria (if any)
3. SQL command according to the question

```
SELECT
FROM
```

4. Alternative SQL commands (if any) with adequate explanation

2.3.4 Question number 4

List of students who have a Semester Grade Point Average for the odd semester of 2025/2026 of no less than 3.25! (understand the difference between Semester GPA and Cumulative GPA)

1. Tables needed to answer the query:
2. Join criteria (if any)
3. SQL command according to the question

```
SELECT
FROM
```

4. Alternative SQL commands (if any) with adequate explanation

2.3.5 Question number 5

List of students and the Cumulative GPA they have obtained to date! (courses that have been retaken only need to be counted once)

1. Tables needed to answer the query:
2. Join criteria (if any)
3. SQL command according to the question

```
SELECT
FROM
```

4. Alternative SQL commands (if any) with adequate explanation

REFERENCES